Cultural Norm Classification

A Two-Stage Transformer Pipeline for Norm Detection and Country Attribution

Technical Report

Yash Vardhan

Natural Language Processing
Department of Computer Science and Engineering
Indian Institute of Technology Hyderabad
CS25MTECH14016

Contents

1 Introduction **2**

2 Dataset **2**
2.1 Composition 2
2.2 Splits 3

3 Data Construction Pipeline **4**

4 The Three-Tier Hard-Negative Strategy **5**
4.1 Tier 1 — Template-identical negatives 5
4.2 Tier 2 — Cultural-group descriptive negatives 5
4.3 Tier 3 — Structural and general factual negatives 6

5 Stage 1 — Norm Classifier **6**
5.1 Architecture 6
5.2 Training configuration 7
5.3 Model selection and evaluation 7

6 Stage 2 — Country Attribution **7**
6.1 Label construction 8
6.2 The class-imbalance problem 8
6.3 Hybrid rule-plus-model inference 9
6.4 Stage-2 training configuration 9

7 Threshold Calibration **9**
7.1 Motivation 9
7.2 Procedure 10
7.3 Deployed operating point 10

8 Error Analysis **10**

9 Results **11**
9.1 Stage 1 — norm classification 11
9.2 Threshold calibration 12
9.3 Stage 2 — country attribution 12

10 Deployment **12**
10.1 Inference service 12
10.2 API surface 13

11 Limitations and Future Work **14**

12 Conclusion **14**

A CLI Reference **15**

B Output Artefacts **15**

C Source Datasets **15**

1 Introduction

Distinguishing a *cultural norm* from an ordinary declarative sentence is deceptively hard. Both can name a country, both can describe human behaviour, and both can share an identical surface form. Consider:

Sentence	**Label**
"In restaurants, Americans tip 15–20 percent of the bill."	Norm
"In restaurants, Americans do not tip service staff."	Non-norm
"In a restaurant, people pay the bill before leaving."	Norm
"In a restaurant, people walk out without paying."	Non-norm
"In Japan, the Meiji Restoration began in 1868."	Non-norm

No reliable lexical or structural cue separates these pairs. A model that learns the rule *"sentences beginning with In {Country}, are norms"* will score well on a naively constructed dataset and fail completely in deployment. The difficulty of this task is therefore a property of the *data*, not the architecture, and the majority of this report is concerned with how the corpus was built so that the shortcut is unavailable.

This report documents the pipeline implemented across six Python modules in `src/` and a containerised web service in `webapp/`. The system:

1. Builds a balanced 50,524-sentence norm/non-norm corpus from nine public sources using a three-tier hard-negative design (`data_prep.py`).
2. Fine-tunes three transformer backbones — DeBERTa-v3, BERT and RoBERTa — for binary norm detection, with large-model variants available (`transformer_model.py`, `large_models.py`).
3. Attributes detected norms to one of 56 countries using a hybrid rule-plus-model classifier (`country_classifier.py`).
4. Calibrates the decision threshold on the validation split to trade recall for precision (`threshold_tuning.py`).
5. Performs structured error analysis by source, template and confidence (`error_analysis.py`).
6. Serves the trained pipeline through a FastAPI backend, a React frontend and an Nginx reverse proxy (`webapp/`, `docker-compose.yml`).

2 Dataset

2.1 Composition

The merged corpus `data/merged_full.csv` contains **50,524 sentences** in exact 1:1 class balance (25,262 norms, 25,262 non-norms), drawn from twelve source streams across nine underlying datasets. Each row carries four fields: `sentence`, `label` $\in \{0, 1\}$, `cultural_group` (1,632 distinct raw values) and `source`.
Sentence length is controlled by a 5–80 word filter applied uniformly to every source. The resulting length distributions are near-identical across classes, so sentence length carries almost no class signal.

Table 1: Corpus composition by source stream. "Tier" refers to the hard-negative taxonomy of Section 4.

Source stream	Class	Rows	Role / Tier
Norm (label = 1) — 25,262 rows			
culturebank_tiktok	Norm	8,822	Templated, agreement ≥ 0.70
normbank	Norm	8,000	Templated from setting / behavior
culturebank_reddit	Norm	5,601	Templated, agreement ≥ 0.70
cultureatlas	Norm	1,903	Country-grounded practice statements
normad	Norm	936	Rule-of-Thumb, Gold Label = yes
Non-norm (label = 0) — 25,262 rows			
wikipedia	Non-norm	9,547	Tier 3 — structural + general facts
normbank_taboo	Non-norm	8,000	**Tier 1** — identical template
squad	Non-norm	4,773	Tier 3 — factual human activity
stereoset	Non-norm	2,071	Tier 2 — group + behaviour, descriptive
crows_pairs	Non-norm	407	Tier 2 — group + behaviour, descriptive
culturebank_reddit_hardneg	Non-norm	247	**Tier 1** — identical template
culturebank_tiktok_hardneg	Non-norm	217	**Tier 1** — identical template

Table 2: Sentence-length statistics by class (words).

Class	Count	Mean	Std	Min	Median	Max
Non-norm (0)	25,262	16.91	10.35	5	14	80
Norm (1)	25,262	17.88	9.62	5	18	80
Overall	50,524	17.40	10.01	5	16	80

2.2 Splits

The corpus is shuffled with a fixed seed (42) and split 80/10/10 with stratification on the label.

Table 3: Stage-1 data splits (stratified, seed = 42).

Split	Rows	Norm (1)	Non-norm (0)
`train.csv`	40,419	20,210	20,209
`val.csv`	5,052	2,526	2,526
`test.csv`	5,053	2,526	2,527
Total	50,524	25,262	25,262

Deduplication on the exact sentence string is applied *within* each source and again across the merged pool before splitting, so no sentence appears in more than one split.

> **Note.** The figures above are computed directly from the committed CSV files. The row counts quoted in `README.md` (36,062 merged / 28,849 train) describe an earlier revision of the corpus and are stale; the README should be updated to match.

3 Data Construction Pipeline

All corpus construction lives in `src/data_prep.py`. The pipeline runs in five stages.

1. Text normalisation. Every candidate sentence passes through `clean_text()`: whitespace is collapsed to single spaces and all non-ASCII characters are stripped. `is_valid()` then enforces the 5–80 word window, which removes both fragments and multi-clause passages that would otherwise dominate the token budget.

2. Norm synthesis from structured fields. CultureBank rows are not sentences; they are records with `context`, `cultural group` and `actor_behavior` fields. `build_norm_sentence()` renders them into prose using *two* alternating templates, assigned deterministically by row parity:

Template A: `"In {context}, {group} {behavior}."`
Template B: `"{group} {behavior} in {context}."`

Using a single template would make sentence-initial *In* a near-perfect predictor of the positive class. The 50/50 alternation removes that cue. The function also guards against degenerate output: if `context` already begins with a preposition (*in*, *at*, *during*, ...) the leading *In* is suppressed under Template A and stripped under Template B, preventing constructions such as "In in Japan".

A deliberate omission is documented in the source: CultureBank's `eval_whole_desc` field was *not* used, because its GPT-generated boilerplate ("...widely regarded as a common practice within the sampled population") is a lexical fingerprint that would trivialise the task.

3. Topic filtering. CultureBank rows are restricted to a 33-topic allowlist (`KEEP_TOPICS`) covering prescriptive behavioural categories — *Social Norms and Etiquette*, *Food and Dining*, *Dress Codes*, *Workplace*, and so on. Three topics flagged as limitations in the CultureBank paper — *Cultural Exchange*, *Migration and Cultural Adaptation*, and *Cultural and Environmental Appreciation* — are excluded because they record reactions and observations (culture shock) rather than prescriptive norms. Critically, the same allowlist is applied to the CultureBank *hard negatives*, so topic cannot separate the classes.

4. Label-noise removal in CultureAtlas. Error analysis on a trained DeBERTa checkpoint revealed that the model was confidently rejecting a subset of CultureAtlas "positive samples" as non-norms — and was correct to do so: the entries were encyclopedic facts, not norms. A twelve-pattern filter (`_CA_BAD_PATTERNS`) was added in response, rejecting sentences that contain a four-digit year, embedded newlines, definitional phrasing (*refers to*, *is defined as*), legislative language (*passed/enacted* near *act/law/bill*), attribution to a named scholar, or colonial-history constructions (*was governed by*). This is a case of the trained model auditing its own training labels.

5. Balancing and splitting. Non-norm sources are consumed in priority order until the norm count is matched; SQuAD fills roughly one third of the residual budget and Wikipedia the remaining two thirds. Both classes are then downsampled to $n = \min(|\text{norms}|, |\text{non-norms}|)$ for exact balance, shuffled in full, and split.

4 The Three-Tier Hard-Negative Strategy

The negative class is the core design of this project. Rather than sampling arbitrary text, non-norms are constructed in three tiers of increasing similarity to the positive class.

4.1 Tier 1 — Template-identical negatives

These are the hardest possible negatives: sentences produced by the *same generator* as the positives.

NormBank taboo (8,000 rows). NormBank rows with `label == 0` mark behaviour that is taboo in a given setting. They are rendered with the byte-identical template used for NormBank norms, `"In a {setting}, people {behavior}."` The only difference between a positive and a negative is the prescriptive polarity of the behaviour:

Label 1	"In a restaurant, people pay the bill before leaving."
Label 0	"In a restaurant, people walk out without paying."

CultureBank low-agreement (464 rows). CultureBank annotates each record with an `agreement` score. Rows at ≥ 0.70 become norms; rows at ≤ 0.20 become negatives, since the source explicitly marks the behaviour as *not* the norm for that group. Both pools pass through the same template renderer, the same topic allowlist and the same length filter — the agreement value is the sole discriminating variable.

This tier is the reason the norm-keyword filter (`NORM_KEYWORDS`: *should*, *must*, *expected to*, *customary*, *polite*, *rude*, ...) is **deliberately not applied** to Tier 1. Those negatives are *supposed* to look like norms.

4.2 Tier 2 — Cultural-group descriptive negatives

Tier 2 closes a narrower gap: sentences that name a cultural group and describe behaviour, but assert a fact rather than an expectation.

- **StereoSet (2,071 rows).** Anti-stereotype continuations are concatenated with their context, giving sentences such as "I know many people who are Russian. Most of them do not drink alcohol." — group named, behaviour described, nothing prescribed.[1]
- **CrowS-Pairs (407 rows).** Only the `sent_less` column is used, and it is filtered twice: a 25-term content filter removes sentences with violent or demeaning language, and a cultural-term regex retains only sentences that actually name a national, ethnic or religious group. Sentences without a group name would add no discriminative value.

[1] The module docstring and the inline comment in `load_stereoset()` disagree on the `gold_label` encoding. The implemented behaviour keeps `gold_label == 0`, which is the anti-stereotype class under the HuggingFace StereoSet schema; the docstring's mapping is stale and should be corrected.

4.3 Tier 3 — Structural and general factual negatives

- **Wikipedia (9,547 rows), split 50/50 into two pools.** Pool A is selected by a regex matching *"In {Capitalised}, {Capitalised} ..."* — sentences with the exact surface form of Template A but purely factual content. Pool B is general encyclopedic text sampled across topics, which prevents the model from learning a topic shortcut.
- **SQuAD (4,773 rows).** Context passages are sentence-split on terminal punctuation followed by a capital letter, then filtered, giving factual descriptions of human activity across many domains.

Tier 3 is where the `NORM_KEYWORDS` filter *is* applied: a Wikipedia sentence containing *should* or *customary* is genuinely ambiguous and is discarded rather than mislabelled.

A source removed on purpose. AG News was dropped from the corpus after inspection: Reuters/AP dateline formatting was a trivially learnable non-norm signal — exactly the kind of shortcut the rest of the design exists to eliminate.

5 Stage 1 — Norm Classifier

5.1 Architecture

Stage 1 is a standard sequence-classification head over a pretrained transformer encoder, instantiated through `AutoModelForSequenceClassification` with `num_labels = 2`. Three base backbones are registered in `transformer_model.py`:

Table 4: Registered Stage-1 backbones.

Key	HuggingFace checkpoint	Approx. params
`deberta`	`microsoft/deberta-v3-base`	∼86 M
`bert`	`bert-base-uncased`	∼110 M
`roberta`	`roberta-base`	∼125 M

A parallel module `large_models.py` registers the large variants (`deberta-v3-large` ∼400 M, `roberta-large` ∼355 M, `bert-large-uncased` ∼340 M) with memory-adjusted hyperparameters, writing to a separate `results/large_models/` tree so that base and large results never collide.

The training objective is the standard cross-entropy loss over two classes,

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B}\sum_{c\in\{0,1\}} y_{i,c}\,\log\frac{\exp(z_{i,c})}{\sum_{c'}\exp(z_{i,c'})}, \tag{1}$$

where z_i are the logits for sentence i and y_i is its one-hot label. Because the corpus is exactly balanced, no class weighting is applied.

Table 5: Stage-1 hyperparameters. "Large" column applies to `large_models.py`.

Hyperparameter	**Base**	**Large**
Max token length	128	128
Batch size (per GPU)	16	8
Gradient accumulation	2	4
Effective batch size	32	32
Max epochs	10	4
Learning rate (AdamW)	2×10^{-5}	1×10^{-5}
Warm-up ratio	0.10	0.10
Weight decay	0.01	0.01
LR schedule	linear w/ warm-up	linear w/ warm-up
Gradient clipping (max norm)	1.0	1.0
Mixed precision (fp16)	enabled	enabled
Early-stopping patience	2 epochs	2 epochs
Random seed	42	42

5.2 Training configuration

Three details in the implementation are worth recording:

- **Effective batch size is held constant at 32** across base and large models. Large checkpoints halve the per-GPU batch to fit in VRAM and double gradient accumulation to compensate, so the optimisation trajectory remains comparable across model scales.
- **The learning rate is halved for large models** ($2\times10^{-5} \rightarrow 1\times10^{-5}$), a standard stabilisation for large-encoder fine-tuning.
- **DeBERTa-v3 weights are explicitly cast to FP32 after loading** (`.float()`). Some DeBERTa-v3 checkpoints ship parameters in FP16, which silently breaks `GradScaler` when mixed-precision training is enabled. This is a real bug that the cast prevents.

5.3 Model selection and evaluation

Selection is on **macro F1 on the validation split**, not accuracy: the checkpoint is written only when validation macro F1 improves, and training halts after two epochs without improvement. The best checkpoint is then reloaded from disk and evaluated once on the held-out test split, so the reported test metrics are never used for selection.

Reported metrics are accuracy and macro-averaged F1, precision and recall. Macro averaging is the correct choice here even under exact class balance, because it prevents a model that is strong on one class and weak on the other from hiding behind a favourable class prior.

6 Stage 2 — Country Attribution

Stage 2 answers a second question: given a sentence that *is* a norm, which country does it belong to? The full inference path is

`sentence` → NormClassifier → *is_norm?* → `yes:` CountryClassifier → country + confidence; `no:` country = `None`

6.1 Label construction

Country labels are not annotated in the source data; they are derived from the free-text `cultural_group` field through a hand-curated normalisation table, `RAW_TO_COUNTRY`, with **203 entries**. The table collapses demonyms, regional identities and sub-national groups onto canonical country names — *Americans*, *Californians*, *New Yorkers*, *Texans*, *Midwesterners* and *Minnesotans* all map to *United States*. Anything absent from the table (pan-regional, religious or generic identities such as *Western*, *Muslim*, *unknown*) is treated as unmappable and dropped rather than guessed.

Classes with fewer than `min_samples = 30` labelled norms are then discarded as too sparse for reliable classification.

Table 6: Stage-2 dataset construction, computed from `data/merged_full.csv`.

Stage	**Sentences**	**Distinct countries**
All norms in corpus	25,262	—
After `RAW_TO_COUNTRY` mapping	14,025	97
After `min_samples` ≥ 30 filter	**13,401**	**56**
Dropped (unmappable group or rare class)	11,861	—

The surviving 13,401 sentences are split 80/10/10 with stratification on the country label, giving **10,720 train / 1,340 validation / 1,341 test**.

6.2 The class-imbalance problem

Stage 2 is a severely imbalanced 56-class problem. The largest class, *United States*, holds 4,192 sentences — 31.3% of the entire dataset — while the smallest surviving classes hold barely above the 30-sample floor. The head-to-tail ratio is **127:1**.

Table 7: Country class distribution: the ten largest and five smallest surviving classes.

Largest classes	**n**	**Smallest classes**	**n**
United States	4,192	Nigeria	37
United Kingdom	835	Angola	36
Germany	756	Israel	34
Canada	478	Czech Republic	34
China	473	Croatia	33
Australia	464		
Italy	450		
France	396		
South Korea	319		
Netherlands	309		

This imbalance is the dominant constraint on Stage 2 and is the reason the primary reported metric is **macro F1** rather than accuracy: a degenerate classifier that always predicts *United States* would score 31.3% accuracy while being useless.

6.3 Hybrid rule-plus-model inference

A key optimisation in `NormCountryPipeline.predict()` is that the neural country classifier is not always invoked. Many norms name their country explicitly in a syntactically recoverable position, and a regular-expression extractor (`extract_country_from_sentence()`) resolves those directly.

The extractor applies three subject-position patterns in order:

1. **Locative setting** — *"In {Country}, ..."* or *"In {Country} people ..."*
2. **Demonym subject** — *"{Demonym} people / culture / society / families ..."*
3. **Sentence-initial fallback** — a direct *"in {country}"* prefix match.

Candidate spans are matched against a token table (the 203-entry demonym map plus a 100-entry plain-country-name table) sorted longest-first, so *South Korea* is matched before *Korea* and *United Kingdom* before *United*. The extractor is written to be conservative: it returns `None` on any ambiguity and defers to the model.

We measured the extractor's behaviour directly on the Stage-2 test split:

Table 8: Rule-based extractor coverage and precision, measured on the 1,341 sentence Stage-2 test split (static evaluation, no model required).

Metric		Value
Sentences where the rule fires (coverage)	273 / 1,341	(20.4%)
Agreement with the gold country when it fires		**96.7%**
Coverage over all 25,262 norms in the corpus		11.5%

This is a favourable trade. One fifth of Stage-2 queries are resolved by a regex at 96.7% precision and effectively zero compute, and the transformer is reserved for the genuinely ambiguous four fifths. When the rule fires, the returned confidence is set to 1.0 to mark it as rule-derived rather than a calibrated model probability — a distinction worth preserving in any downstream consumer of the API.

6.4 Stage-2 training configuration

Stage 2 reuses the Stage 1 training machinery with a wider classification head (56 outputs) and a shorter schedule: max 5 epochs, batch size 16, gradient accumulation 2, learning rate 2×10^{-5}, patience 2, seed 42. Each backbone writes its own checkpoint directory together with a `label_map.json` recording the `id` → `country` mapping, which is required at inference time and is the reason a checkpoint without it is rejected at pipeline start-up.

7 Threshold Calibration

7.1 Motivation

Argmax decoding of a two-class softmax is equivalent to thresholding $P(\text{norm})$ at 0.5. The recorded run notes for the DeBERTa checkpoint document a specific failure mode at that default: the model produced **877 false positives**, and those false positives had a **mean $P(\text{norm})$ of only 0.54**. The errors were not confident mistakes — they were the

model barely tipping over the decision boundary on Tier-1 NormBank taboo sentences, exactly the negatives designed to be hardest.

Meanwhile the norm class was being recalled at approximately **0.99**. The classifier was not weak; it was *miscalibrated for the operating point we wanted*. That is a threshold problem, not a training problem, and it is far cheaper to fix.

7.2 Procedure

`threshold_tuning.py` implements the standard calibration protocol:

1. Run the trained checkpoint over the validation split *once*, caching $P(\text{norm})$ for every sentence.
2. Sweep the decision threshold τ from 0.30 to 0.80 in steps of 0.01 (51 operating points), computing accuracy, macro F1, per-class F1, per-class precision and recall, and raw false-positive / false-negative counts at each.
3. Select $\tau^\star = \arg\max_\tau \text{macro-F1}_{\text{val}}$.
4. Apply $\tau^\star$ *unchanged* to the test split and report against the $\tau = 0.50$ baseline.

Because probabilities are cached before the sweep, all 51 operating points cost a single forward pass over the split. Critically, the threshold is selected on validation data and only then applied to test — the test split never participates in selection.

7.3 Deployed operating point

The web service ships with a default threshold of **0.6** rather than 0.5 (`PredictRequest.norm_threshold` and the `--norm_threshold` default in `country_classifier.py`), reflecting the calibration result. The threshold is exposed as a request parameter, so a caller can move along the precision/recall curve per query without redeploying.

Table 9: Threshold trade-off. Direction of movement is determined by the architecture of the sweep; exact magnitudes come from `results/threshold_tuning/threshold_metrics.csv` after a run.

Raising τ	**Effect**	**Cost**
$0.50 \to 0.60$	Removes borderline FPs (mean conf. 0.54)	Loses genuine norms scoring in $[0.50, 0.60)$
$0.60 \to 0.80$	Non-norm precision continues to rise	Norm recall degrades sharply from ≈ 0.99
Below 0.50	Norm recall approaches 1.0	FP count grows past the 877 baseline

8 Error Analysis

`error_analysis.py` exists to answer *why* the model fails, not just how often. It runs the trained checkpoint over the test split, isolates every false positive and false negative, and decomposes them along four axes.

1. **By source.** Which of the twelve source streams generate errors? This directly tests the hard-negative hypothesis: if Tier-1 sources (`normbank_taboo`, `culturebank_*_hardneg`) dominate the false positives, the tiering worked as designed — those are the negatives that are *supposed* to be hard. If Tier-3 Wikipedia sentences dominate instead, something more basic is wrong.

2. **By template.** Errors are bucketed by surface form (`detect_template()`): Template-A (*In* {*ctx*}*,* {*grp*} {*behav*}), Template-B ({*grp*} {*behav*} *in* {*ctx*}), or Other. A model that has genuinely escaped the structural shortcut should show errors spread across templates rather than concentrated in one.

3. **By confidence.** The mean confidence of each error class is reported. As Section 7 shows, this is the diagnostic that motivated threshold tuning: low-confidence errors are a calibration problem, high-confidence errors are a learning problem, and they call for entirely different fixes.

4. **By sentence length and cultural group.** Length distributions of errors are compared against correct predictions, and errors are broken down by `cultural_group` to expose any per-culture bias in the classifier.

The module writes `false_positives.csv`, `false_negatives.csv`, a readable `error_summary.txt`, and five diagnostic plots. As documented in Section 3, this analysis has already produced one concrete corpus fix — the CultureAtlas encyclopedic-entry filter — which is the clearest possible evidence that the error loop is closing.

9 Results

Note on reported numbers. The dataset statistics, the rule-extractor measurements in Section 6.3, and every configuration value in this report were computed or read directly from the committed code and data. The trained checkpoints (`saved_models/`) and the metrics tree (`results/`) are not part of the repository, so the model-performance cells below are marked — and must be populated from `results/*_results.json` after a training run. The two recorded observations that *are* quoted — 877 false positives at $\tau = 0.50$ with mean confidence 0.54, and norm recall ≈ 0.99 — come from the run notes preserved in `threshold_tuning.py`.

9.1 Stage 1 — norm classification

Table 10: Stage-1 test-set results, base backbones. Populate from `results/metrics_comparison.csv`.

Model	Accuracy	Macro F1	Precision	Recall
DeBERTa-v3-base	—	—	—	—
BERT-base-uncased	—	—	—	—
RoBERTa-base	—	—	—	—

Table 11: Stage-1 test-set results, large backbones. Populate from `results/large_models/large_models_comparison.csv`.

Model	Accuracy	Macro F1	Precision	Recall
DeBERTa-v3-large	—	—	—	—
RoBERTa-large	—	—	—	—
BERT-large-uncased	—	—	—	—

Table 12: Test-set comparison at the default versus tuned threshold. Populate from `results/threshold_tuning/threshold_summary.txt`.

Threshold	Accuracy	Macro F1	Norm rec.	Non-norm rec.	FP	FN
$\tau = 0.50$ (default)	—	—	≈ 0.99	—	877	—
$\tau = \tau^\star$ (tuned)	—	—	—	—	—	—

9.2 Threshold calibration

9.3 Stage 2 — country attribution

Table 13: Stage-2 test-set results over 56 classes. Populate from `results/country/country_metrics_comparison.csv`. Macro F1 is the headline metric given the 127:1 class imbalance.

Backbone	Accuracy	Macro F1	Weighted F1
DeBERTa-v3-base	—	—	—
BERT-base-uncased	—	—	—
RoBERTa-base	—	—	—

10 Deployment

The trained pipeline is served as a three-container application orchestrated by `docker-compose.yml`.

10.1 Inference service

Four design decisions in `webapp/backend/` are worth recording:

- **Models are preloaded at start-up, not per request.** A `startup` hook calls `preload_all()`, which populates a module-level `_ModelCache` with the norm model and every available country model. Per-request latency therefore excludes checkpoint loading. The compose healthcheck grants a 60-second `start_period` to accommodate this.
- **Inference runs on CPU** (`DEVICE = torch.device("cpu")`), so the service deploys on commodity hardware with no CUDA runtime.

Table 14: Stage-2 hybrid inference: measured directly, no trained model required.

Quantity	Value
Test queries resolved by the rule extractor	20.4%
Rule-extractor precision when it fires	96.7%
Test queries requiring a transformer forward pass	79.6%

Table 15: Service topology.

Service	Stack	Role
`backend`	FastAPI + PyTorch	Model inference, metrics API
`frontend`	React + Vite	Classifier, Metrics and About pages
`nginx`	Nginx	Reverse proxy, static plot serving

- **HuggingFace offline mode is forced before `transformers` is imported.** `TRANSFORMERS_OFFLINE` and `HF_HUB_OFFLINE` are set at the top of `inference.py`, above the imports, because recent `huggingface_hub` versions validate absolute local paths as repository IDs and break checkpoint loading inside the container. Import order is load-bearing here.
- **Stage 2 runs all three backbones in parallel and returns all three answers.** Rather than picking a winner, `/api/predict` returns the top-3 countries from each of the DeBERTa, BERT and RoBERTa country classifiers, letting the interface present model agreement as a confidence signal in its own right. Stage 1 uses BERT alone.

10.2 API surface

Table 16: HTTP endpoints exposed by the FastAPI backend.

Method	Path	Purpose
GET	`/api/health`	Liveness probe for the compose healthcheck
GET	`/api/models`	Which checkpoints are present on disk
POST	`/api/predict`	Two-stage prediction; body carries `sentence` and `norm_threshold`
GET	`/api/results`	Bundled metrics JSON for both stages
GET	`/api/results/plots`	List available diagnostic plots
GET	`/api/results/plots/{f}`	Serve one training curve or confusion matrix

Model weights and the results tree are mounted **read-only** (`:ro`) into the containers, so a running service cannot corrupt the artefacts it serves.

11 Limitations and Future Work

- **Templated positives remain a distribution risk.** Roughly 22,400 of the 25,262 norms are synthesised from two fixed templates. The Tier-1 negatives use the same templates, which neutralises the shortcut *within* this corpus, but the model's behaviour on free-form norms written by humans in the wild is not directly measured by this test split.
- **Country attribution inherits a severe head bias.** The United States alone accounts for 31.3% of Stage-2 data and the head-to-tail ratio is 127:1. Class-balanced loss, focal loss or per-class resampling are the obvious next steps; none is currently applied.
- **Genuinely cross-cultural norms have no correct single label.** A norm such as *"guests remove their shoes before entering"* is shared across dozens of countries, but Stage 2 is a single-label classifier and is forced to choose. Multi-label attribution, or an explicit *shared / pan-cultural* class, would model this honestly.
- **The 41 countries below the 30-sample floor are silently unreachable.** The pipeline cannot predict them at all, and the API does not surface that limitation to callers.
- **Rule-extractor confidence is not comparable to model confidence.** Rule hits return 1.0 by construction. Any downstream consumer that ranks or thresholds on `country_confidence` will systematically over-trust rule-derived answers; the field should carry a provenance flag.
- **Two documentation defects should be corrected**: the stale row counts in `README.md` (Section 2.2) and the contradictory `gold_label` mapping in the `load_stereoset()` docstring (Section 5.2). Neither affects the produced artefacts, but both mislead a reader of the code.

12 Conclusion

This report has documented an end-to-end cultural-norm classification pipeline, from raw source ingestion through to a containerised HTTP service. The contributions are:

1. **A 50,524-sentence balanced corpus with an adversarially constructed negative class.** Nine public datasets are merged into three tiers of hard negatives, the hardest of which are rendered by the *same* template as the positives and differ only in prescriptive meaning. Two alternating templates, a shared topic allowlist across both classes, and the deliberate removal of AG News and CultureBank's GPT-generated boilerplate close the structural shortcuts that would otherwise make the task trivial.
2. **A two-stage classification pipeline over three interchangeable transformer backbones**, with base and large variants, macro-F1 model selection, early stopping and a reproducible fixed-seed configuration throughout.

3. **A hybrid rule-plus-model country attribution stage** that resolves 20.4% of test queries by regex at 96.7% precision and reserves the 56-class transformer for the ambiguous remainder.
4. **A calibration and diagnosis loop.** Threshold sweeping converts a 0.99-recall classifier into a precision-balanced one without retraining, and structured error analysis by source, template and confidence has already fed one concrete correction back into the corpus — the CultureAtlas encyclopedic-entry filter, discovered because the trained model was right and the training labels were wrong.
5. **A deployable service**: preloaded CPU inference, read-only weight mounts, a health-checked compose topology, and a three-backbone Stage-2 response that surfaces model disagreement instead of hiding it.

A CLI Reference

```
# ---- Stage 0: build the corpus ----
python src/data_prep.py

# ---- Stage 1: norm classifier ----
python src/transformer_model.py --model deberta
python src/transformer_model.py --model all
python src/transformer_model.py --model bert --epochs 5 --batch_size 8

# Large variants (separate results tree)
python src/large_models.py --model deberta_large
python src/large_models.py --model deberta_large --batch_size 4 --grad_accumulation 8

# ---- Stage 2: country classifier ----
python src/country_classifier.py --mode train --model all
python src/country_classifier.py --mode eval
python src/country_classifier.py --mode predict \
  --norm_model deberta --model deberta --norm_threshold 0.6 \
  --sentences "In Japan, people bow when greeting someone." "The sky is blue."

# ---- Diagnostics ----
python src/threshold_tuning.py --model deberta --t_min 0.30 --t_max 0.80 --t_step 0.01
python src/error_analysis.py --model deberta --n_samples 50

# ---- Serve ----
docker compose up --build
```

B Output Artefacts

C Source Datasets

NormBank norms and taboos are both capped at 8,000 rows and drawn from the train split only, preventing NormBank from dominating either class and avoiding contamination from its own held-out data.

Table 17: Files produced by a full pipeline run.

Path	**Contents**
`data/merged_full.csv`	Full balanced corpus, 50,524 rows
`data/{train,val,test}.csv`	Stratified 80/10/10 splits
`saved_models/{model}_best/`	Best Stage-1 checkpoint per backbone
`saved_models/country_{model}_best/`	Stage-2 checkpoint + `label_map.json`
`saved_models/large_models/`	Large-variant checkpoints
`results/{model}_results.json`	Stage-1 metrics, config and per-epoch history
`results/metrics_comparison.csv`	Stage-1 backbones side by side
`results/plots/`	Training curves, confusion matrices, comparison chart
`results/country/`	Stage-2 metrics, plots, comparison CSV
`results/large_models/`	Large-variant metrics and plots
`results/threshold_tuning/`	Sweep curves, before/after confusion matrices, `threshold_metrics.csv`, `threshold_summary.txt`
`results/error_analysis/`	`false_positives.csv`, `false_negatives.csv`, `error_summary.txt`, 5 diagnostic plots

Table 18: Underlying public datasets and the role each plays.

Dataset	**Used for**	**Selection criterion**
CultureBank (Reddit)	Norm + Tier-1 neg	agreement ≥ 0.70 / ≤ 0.20
CultureBank (TikTok)	Norm + Tier-1 neg	agreement ≥ 0.70 / ≤ 0.20
NormBank	Norm + Tier-1 neg	label $\in \{1, 2\}$ / label $= 0$; train split only
NormAd	Norm	Gold Label = *yes*
CultureAtlas	Norm	`positive_sample`, 12-pattern filter
StereoSet	Tier-2 negative	anti-stereotype continuations
CrowS-Pairs	Tier-2 negative	`sent_less`, safety + cultural filter
SQuAD	Tier-3 negative	contexts, sentence-split
Wikipedia	Tier-3 negative	50% structural pool, 50% general
AG News	*removed*	dateline formatting was a trivial shortcut